



UNIVERSITÀ DI PAVIA

Optimization of a 2D Convolution

Advanced Computer Architecture

Department of Electrical, Computer and Biomedical Engineering

Nicolò Galli
September, 2026

https://github.com/nico-log/2D_Conv_Parallel.git

Abstract

This document reports the implementation of a 2D Convolution CUDA kernel, a project for the Advanced Computer Architecture course. A serial CPU implementation has been used as a baseline for computing speed-up, then starting from a naive GPU implementation, various optimizations were applied to get an efficient solution.

Keywords: image processing, image filtering, convolution, CUDA, GPU.

Contents

1	Introduction	1
2	Convolution Operation	1
3	Padding	1
4	Filters	2
5	Program features	2
6	CPU implementation	3
7	GPU implementation	3
7.1	Naive Implementation	4
7.2	Shared Memory	4
7.3	Constant Memory	5
7.4	Vectorized Accesses	6
7.5	Loop unrolling	6
8	Benchmark	7
9	Results	8
10	Roofline model	9
11	Conclusion	11
12	Future Developments	12

1 Introduction

The goal of the project is to implement a CUDA kernel that applies a 3×3 convolutional filter to an image and demonstrate the effectiveness of parallel computing. The project compares a CPU implementation with different GPU implementations, measuring the execution times and the speed-up. The problem was initially addressed using a naive implementation, then different advanced techniques have been applied to get better performance and efficiency.

Even if different optimized solutions already exist, building a kernel of this type from scratch has significant educational value. Furthermore, having a customized solution allows applying custom filters or combining filters for specific domains without dependencies on large external libraries, which can be useful in embedded systems. This solution can be applied in different domains, such as real-time computer vision applications.

2 Convolution Operation

The convolution is an operator that transforms an input signal $\mathbf{x}$ into another signal $\mathbf{z}$. Consider a one dimensional signal $\mathbf{x} = (x_0, x_1, \dots, x_{n-1})$, each output value z_u is obtained as a weighted sum of a group of k consecutive input values $x_u, x_{u+1}, \dots, x_{u+k-1}$:

$$z_u = \sum_{s=0}^{k-1} w_s \cdot x_{u+s} \quad (1)$$

where $\mathbf{w} = (w_0, w_1, \dots, w_{k-1})$ is a vector of weights called *filter* or *kernel*.

Generally, color images are represented as 2D signals with 3 channels (RGB). Given:

- $\mathbf{x}$: input image ($height \times width \times 3$)
- $\mathbf{w}$: filter matrix ($k \times k$)

The convolution $\mathbf{z} = \mathbf{x} * \mathbf{w}$ is computed as follows¹:

$$z_{u,v,c} = \sum_{t=0}^{k-1} \sum_{s=0}^{k-1} w_{s,t} \cdot x_{u+s,v+t,c} \quad (2)$$

The same $k \times k$ spatial filter is applied *independently* to each color channel. The 2D convolution can be explained in a more practical way as shown in Figure 1.

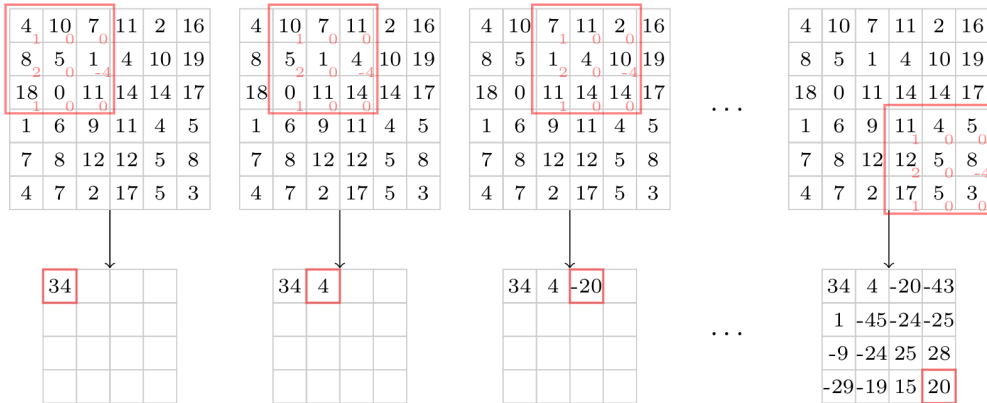


Figure 1: Example of a 2D single-channel convolution with a 6×6 input image, a 3×3 filter resulting in a 4×4 output image

3 Padding

The output image produced by the convolution operation is smaller than the input image. Given an input image $height \times width$ and a filter $k \times k$, the convolution would result in an output of dimension $(height - k + 1) \times$

¹Cross-correlation formula commonly used in ML frameworks, not the mathematical definition

($width - k + 1$). The padding technique allows controlling the output image size by enlarging the input image. In detail, padding consists of adding a frame of additional pixels, usually set to zero (*zero-padding*), around the border of the input image. A common padding strategy, that has been applied in this work, is the same size convolution. Same size convolution padding ensures that the output image size matches the input image size. For odd values of k , this corresponds to a padding of $(k - 1)/2$ on each side. For $k = 3$ the padding is 1.

0	0	0	0	0	0	0
0	x_{00}	x_{01}	x_{02}	x_{03}	x_{04}	0
0	x_{10}	x_{11}	x_{12}	x_{13}	x_{14}	0
0	x_{20}	x_{21}	x_{22}	x_{23}	x_{24}	0
0	x_{30}	x_{31}	x_{32}	x_{33}	x_{34}	0
0	x_{40}	x_{41}	x_{42}	x_{43}	x_{44}	0
0	0	0	0	0	0	0

Figure 2: Padded input image

z_{00}	z_{01}	z_{02}	z_{03}	z_{04}
z_{10}	z_{11}	z_{12}	z_{13}	z_{14}
z_{20}	z_{21}	z_{22}	z_{23}	z_{24}
z_{30}	z_{31}	z_{32}	z_{33}	z_{34}
z_{40}	z_{41}	z_{42}	z_{43}	z_{44}

Figure 3: Same size convolution output

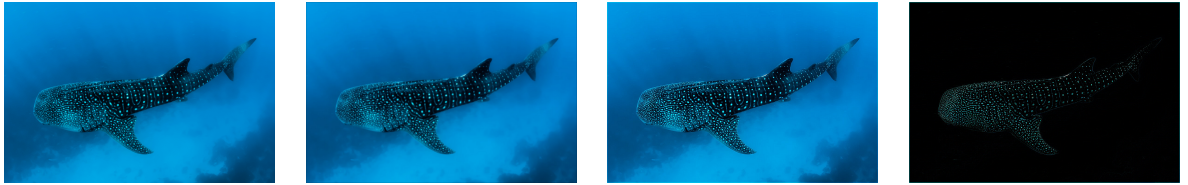
Note that applying zero-padding introduces an artificial high-contrast step at the image borders. Edge and sharpen filters interpret this discontinuity as an edge, producing a characteristic bright perimeter frame in the output images.

4 Filters

In image processing convolutions are used to obtain a large variety of effects and the choice of the filter obviously affects the output image. In this work three types of filters have been implemented:

- **Blur Filter:** removes noise and details from the original image
- **Sharpen Filter:** boosts the edge contrast
- **Edge Filter:** marks boundaries of the objects in the original image.

The filter size is fixed: 3×3 .



Test Image

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Blur

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Edge

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

5 Program features

The program is implemented as a command-line tool. It requires four parameters:

```
./convolution <filter> <iterations> <input_image_path> <output_image_path>
```

1. the filter type (edge, sharpen, blur)
2. the number of iterations for benchmarking (see Section 8)
3. the input image path
4. the output image path

The program is composed of a CPU implementation of the 2D convolution algorithm and five GPU implementations, starting from a naive one, introducing one optimization at each step:

0. CPU implementation (baseline)
1. GPU naive version
2. GPU shared memory
3. GPU shared memory + constant memory
4. GPU shared memory + constant memory + vectorization
5. GPU shared memory + constant memory + vectorization + loop unrolling

The program prints: the CPU execution time (used as baseline) and the CPU throughput (MPixel/s); then for each GPU implementation prints: the average execution time, the throughput (MPixel/s) and the resulting speed-up.

Each implementation produces an output image. A key feature for checking the correctness of the produced images is the *validation function*. The function compares two images pixel by pixel and returns an integer value used as a success/failure flag. Each GPU output image is compared with the CPU output image, which is always used as the reference. If all GPU outputs pass the validation, only a single output image is saved; otherwise, all output images are saved. The validation function accepts a tolerance value, fixed to one in this project. Therefore, two corresponding pixel values are considered equivalent if their absolute difference does not exceed one intensity level. This tolerance accounts for small numerical differences that may arise from floating-point arithmetic, rounding, casting, and truncation.

To read and write the images, the program uses two external libraries: `stb_image.h` and `stb_image_write.h`. These efficient libraries are suitable for the project since they are *header-only*. Libraries of this type can be included in the project by defining the corresponding implementation macros. This avoids the need to compile and link separate library source files. When the input image is read, the library decodes the input image and loads the pixels into RAM returning a pointer to a single flattened 1D array. Since we have four channels (RGBA), the array has the following structure: [R, G, B, A, R, G, B, A, ...]

The virtual alpha channel (A) has been added to the original RGB space to exploit the vectorized accesses optimization (see Section 7).

6 CPU implementation

After parsing the command-line arguments, the CPU convolution function can be launched tracking the execution time. The serial CPU implementation consists of several for-loops. Three nested loops are necessary to iterate over each pixel in the image (width, height, channel). For each pixel two nested loops are necessary to iterate over the filter matrix to compute the output pixel through the convolution. As shown in Figure 1, the coefficients of the filter are overlaid in the upper left corner of the input image, each pixel is multiplied by the corresponding filter element and the sum of the products is taken as the first output element. Then the filter is moved one step to the right and the second element is computed. The process is repeated by placing the filter on the input image in each possible position until all the output elements have been computed. The zero padding is applied implicitly: if the accessed pixel is out of the image boundary, the pixel value is set to zero. At the end, the convolution result is clamped between 0 and 255, and written to the corresponding position of the output pixel. Notice that the channel loop iterates over 3 channels, the alpha channel is automatically set to 255 (fully opaque).

7 GPU implementation

As already mentioned, the parallel part of the code is composed of five different CUDA kernels. Each one introduces one optimization, but the initialization of *grid* and *blocks* is the same. The size of a single block has been set to 16×16 , so each block has 256 threads.

```
threadsPerBlock(16, 16)
```

Bidimensional blocks are suitable for image processing applications because there is a match between the pixel and the threads organization. The grid size, i.e. the number of blocks, depends on the input image size. The ceiling operation makes it possible to cover all pixels even if the input image dimensions are not a multiple of the block size. The ceiling operation allows rounding up the number to the next multiple of the block size:

```
numBlocks((width + threadsPerBlock.x - 1) / threadsPerBlock.x,  
          (height + threadsPerBlock.y - 1) / threadsPerBlock.y)
```

Each thread is uniquely identified by the following coordinates:

```
x = blockIdx.x * blockDim.x + threadIdx.x
y = blockIdx.y * blockDim.y + threadIdx.y
```

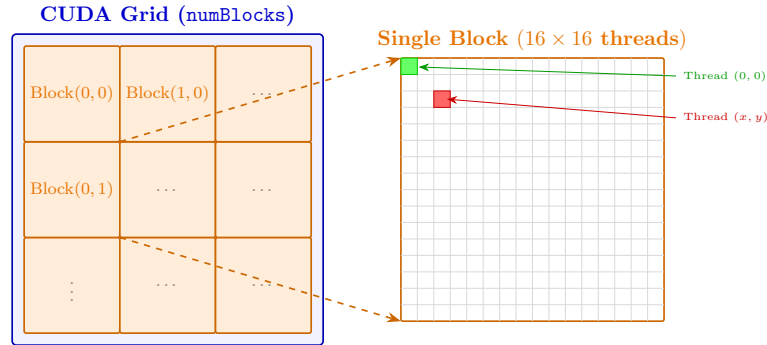


Figure 4: Grid and blocks organization

Each of the following kernels uses the GPU memory hierarchy in a different way, in any case it is possible to identify a common flow to manage the information exchange between the host and the device:

1. The CPU must allocate the correct space in the device memory for the input image and the output image. The allocated space is based on the images size. Notice that it is not possible to overwrite the same space during the computation due to data dependency.
2. The input image is copied from the host memory to the device memory.
3. The kernel is launched, measuring the execution time with `cudaEventRecord()`
4. The output image is copied from the device memory to the host memory.
5. The allocated memory is freed

Notice that the CUDA calls are asynchronous with respect to the host. For this reason it is often difficult to identify what function or routine is responsible for an error. A suitable macro (`CUDA_CHECK(call)`) has been implemented; it wraps CUDA API calls, simplifying the error management. It guarantees the robustness of the code and simplifies the debugging.

7.1 Naive Implementation

The first parallel implementation of the 2D convolution algorithm is based only on the use of *global memory*. The global memory is the primary, largest storage space on a GPU. It is accessible by all threads and the CPU. In the CPU implementation there is only one thread that executes all the computation in sequence. In this first GPU implementation there are multiple threads and each one computes one output pixel, so the algorithm is similar to the CPU implementation but the for-loops dedicated to read the pixel along the width and the height are no longer necessary. A massive speed-up is expected but the performance is limited by the global memory. It can hold massive datasets but is slower to access, and to compute the convolution each thread reads 9×3 values (nine filter elements and three channels) from the global memory, so the same pixel of the input image is read repeatedly by different threads.

7.2 Shared Memory

This first optimization introduces the use of *shared memory*. The shared memory is an on-chip programmable memory, private for each block and shared for each thread in a block. It has a higher bandwidth and a lower latency than global memory. The kernel is divided into two phases:

1. The input image is copied from global to shared memory
2. The pixels are read from the shared memory and the convolution operation is computed

Even if it seems a long and more complex process, the shared memory usage reduces the global memory accesses, so a lower execution time is expected. In this context, copying the data from global to shared memory could be a delicate process that requires close attention. Each block works on $16 \times 16 = 256$ pixels but the threads near the border need pixels outside the block to compute the convolution (halo pixels). For this reason an 18×18 shared tile must be allocated for each block. Each block has 256 threads but $18 \times 18 = 324$ pixels must be loaded for each tile:

1. the first 256 threads load the first 256 pixels
2. the first 68 threads load the remaining 68 pixels

Since the input image has three channels, each thread executes conceptually three global memory accesses; $324 \times 3 = 972$ elements are transferred from global to the 3D shared memory tile ($18 \times 18 \times 3$). At the end of the first phase, the kernel uses `__syncthreads()` to ensure that all threads have completed the loading phase before starting the convolution.

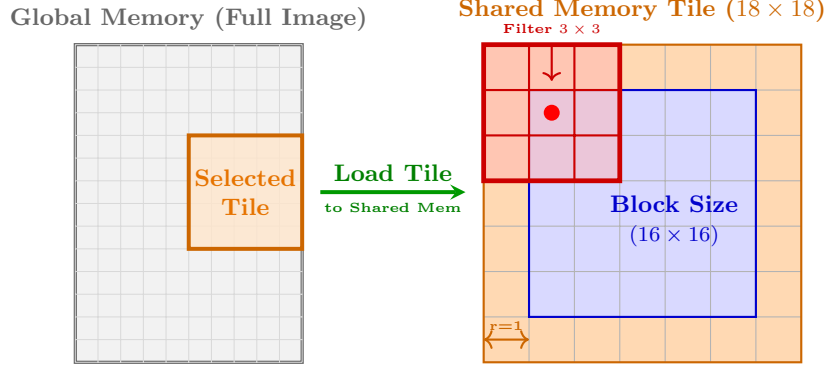


Figure 5: Pixels transferring from global to shared memory

In the second phase of the kernel the convolution operation is computed. The algorithm is similar to the previous implementation with the difference that pixel values are no longer read from the global memory but from the shared memory with a clear advantage in terms of latency. In the convolution operation, it is necessary to read both the input image pixels and the filter matrix elements. While the pixels are loaded in shared memory and their access is fast, the filter matrix elements are still in global memory.

7.3 Constant Memory

The filter matrix elements are read many times, storing them in the global memory causes a bottleneck. This optimization introduces the use of *constant memory*. The constant memory is a small, read-only part of the global device memory used to store data that does not change during the execution, it is generally limited to 64KB. The filter matrix represents read-only data that occupies only 36 bytes ($3 \times 3 \times 4$ byte), so instead of using the global memory, the CPU can load it into the constant memory. The main advantage of constant memory in this context is its broadcast mechanism, which allows managing multiple accesses to the filter matrix within the same clock cycle. With the filter stored in the global memory, the multiple requests from different threads are scheduled and a significant amount of time could potentially be wasted during this process. The constant memory simplifies this process: since all threads in a warp access the same filter coefficient at each iteration, constant memory can broadcast that coefficient to the warp efficiently during the same clock cycle avoiding redundant memory accesses.

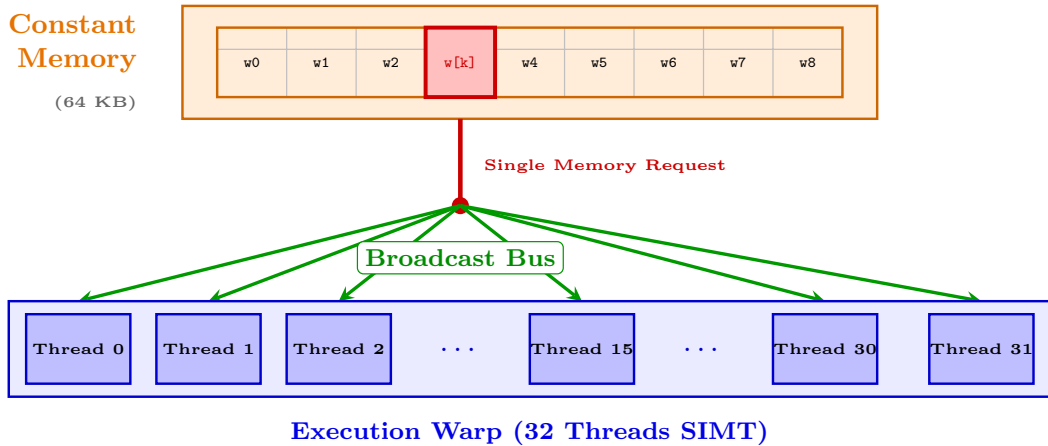


Figure 6: Constant Memory broadcast mechanism.

7.4 Vectorized Accesses

When a 32-thread warp accesses global memory, the hardware tries to coalesce the individual memory requests into the smallest possible number of memory transactions. Coalescing efficiency depends on the address pattern and alignment of the accesses.

In the previous implementations, each RGB channel is accessed separately from the RGBA image. Since the channels are stored interleaved, consecutive pixels are four bytes apart in memory. Therefore, when a warp accesses the same channel of consecutive pixels, the addresses have a regular 4-byte stride rather than being consecutive at byte level. These accesses can still be handled efficiently by the memory system, and the cache hierarchy can further reduce the cost of subsequent accesses. Consequently, the three channel loads performed by a thread do not necessarily result in three independent DRAM accesses. Nevertheless, they are still separate memory-load operations from the point of view of the kernel.

This optimization introduces vectorized memory accesses. The built-in CUDA structure `uchar4` is used so that each thread can access the four bytes corresponding to one complete RGBA pixel at once. Conceptually, this replaces the three separate 1-byte loads required to retrieve the R, G, and B channels with a single 4-byte vectorized load. As a result, consecutive threads in a warp access consecutive `uchar4` elements, matching the four-byte layout of the RGBA image. This improves the access granularity and reduces the number of explicit memory-load operations required by each thread. Consequently, vectorization can also improve the efficiency of memory coalescing, since consecutive threads request consecutive four-byte elements rather than individual bytes belonging to the same-channel positions of different pixels.

The alpha channel is not involved in the convolution computation, but it is loaded together with the RGB channels because `uchar4` represents the complete four-byte RGBA pixel. Therefore, vectorization introduces an additional byte of data transfer per pixel compared with loading only the three required RGB channels. This represents a small bandwidth overhead that is traded for a more appropriate access granularity and fewer explicit memory-load operations.

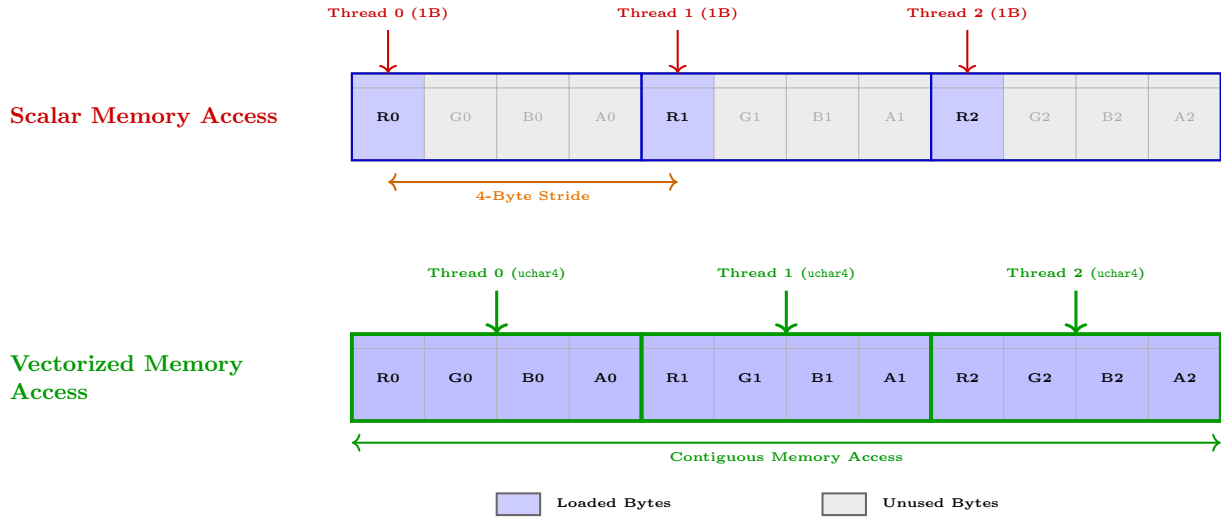


Figure 7: Comparison between scalar and vectorized memory accesses.

7.5 Loop unrolling

In the previous implementation, also the channel for-loop is removed, but the nested loop to iterate over the filter matrix still remains. Each loop can lead to an overhead due to the variable incrementation, the condition check and the branches. This final optimization introduces the *loop unrolling*. Knowing the number of iterations of a loop it is possible to eliminate the loops at assembly-level by copying the loop body. This eliminates the loop overhead and potentially allows instruction reordering. The loop unrolling has been implemented through the `#pragma unroll` directive. Because the loop bounds are compile-time constants and the loop is very small (fixed filter size 3×3), the CUDA compiler (`nvcc`) may already perform substantial optimization. Therefore, explicitly requesting unrolling has little expected impact on the performance.

8 Benchmark

The fundamental measured metric is the *execution time* (ms) of each implementation. Then the CPU execution time is used as baseline to compute the speed-up:

$$SpeedUp = \frac{TimeCPU_{[ms]}}{TimeGPU_{[ms]}} \quad (3)$$

To measure the CPU execution time the standard C++ timer (`std::chrono`) has been used, while measuring of the execution time of each GPU kernel adopts the CUDA Events API `cudaEventRecord()`. The CPU and GPU have different execution and synchronization models. CUDA kernel launches are asynchronous with respect to the host, so directly measuring the host-side time around a kernel launch would not accurately represent the kernel execution time. To obtain an accurate GPU measurement, two CUDA events are recorded in the same CUDA stream: one immediately before the kernel launch and one after it. After synchronization, `cudaEventElapsedTime()` is used to determine the elapsed GPU execution time.

Note that the reported GPU times measure kernel execution only and therefore do not include host-to-device or device-to-host transfers, memory allocation, or other setup costs. The CPU measurement represents the execution time of the complete CPU convolution.

In the image processing and computer vision domains another common metric is the throughput in terms of mega-pixel over seconds:

$$Throughput = \frac{width \times height}{10^6 \times Time_{[s]}} \quad [MP/s] \quad (4)$$

Execution time alone provides incomplete information, as it naturally scales with the input image resolution. In contrast, throughput in Megapixels per second (MP/s) provides a normalized rate metric indicating the volume of data processed per unit of time. This measure is also useful since the image processing algorithms are typically memory bound (See Section 10), so information about the GPU memory bandwidth usage is fundamental.

To ensure stable, repeatable, and less noisy measurements, two standard benchmarking practices were adopted: a *warm-up* phase and *multi-iteration execution*.

The warm-up consists of launching the kernel one time, before measuring the kernel execution time. This procedure is applied to get clean measures reducing the impact of first-launch and initialization overheads. The first kernel launch is more complex than the following ones because of driver initialization, compilation, code loading and possible compulsory misses. For this reason the first launch is not measured, it would produce an outlier.

The multiple kernel execution consists of launching the kernel many times measuring the total execution time. Then, the total execution time is used to compute the average execution time:

$$AvgExecutionTime = \frac{Time_{ms}}{Iterations} \quad [ms] \quad (5)$$

This is done because kernels have a really low execution time therefore they are vulnerable to possible latencies introduced by the system. Moreover, there is a secondary question related to the clock frequency. Modern GPUs dynamically adapt the clock frequency based on the task; the continuous execution of the kernel, combined with one warm-up execution, forces the hardware to exit from the energy saving state increasing the clock frequency to get stable measures at the full possible speed. The warm-up procedure has been applied also in the measurement of the CPU execution time because the first function launch can cause compulsory misses and page faults that do not always occur in the standard use; then because also the CPU dynamically adapts the clock frequency. While, unlike the kernels, the measure of the execution time has been done with only one iteration. Even if also the CPU can be affected by possible latency introduced by the system, the execution time is high enough to ignore this phenomenon, without wasting time during the testing phase. Table 1 reports the benchmark settings used to get the results shown in Section 9. Notice that the number of GPU iterations is not a constant, but it is a CLI parameter.

Implementation	Warm-up	Execution time
CPU	1 iteration	1 iteration
GPUs	1 iteration	100 iterations

Table 1: Benchmark settings

9 Results

Here is some basic information about the system and the hardware used for the measurement:

System	
Driver version	560.64
CUDA version	12.6
Device	
Product	NVIDIA GeForce GT 1030
Memory	2048 MB GDDR5
Memory bus	64-bit
L1 cache	48 KB (per SM)
L2 cache	512 KB

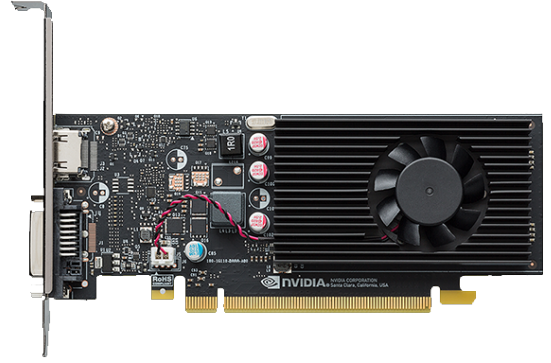


Table 2: Test environment

As previously mentioned, the measurements have been taken adding one optimization at a time, showing the impact of each one. Consequently six execution times have been measured, one CPU implementation, used as baseline to compute the speed-up, and five GPU kernels. The resolution of the input JPEG image used for the test is 4344×2896 with four channels (RGBA). The following tables report the result for each implemented filter.

Kernel	Time (ms)	Throughput (MP/s)	Speedup
CPU (Baseline)	297.55	42.28	1.00x
GPU (Naive)	15.42	815.98	19.30x
GPU (Shared Mem)	10.11	1244.06	29.42x
GPU (+ Constant)	6.98	1802.84	42.64x
GPU (+ Vectorized)	4.83	2605.57	61.63x
GPU (+ Unrolled)	4.81	2613.04	61.80x

Table 3: Performance Summary - **Edge Filter**

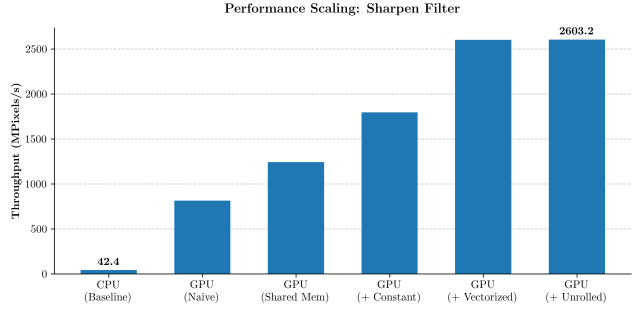
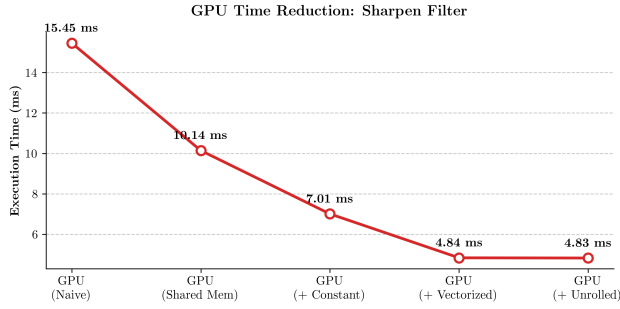
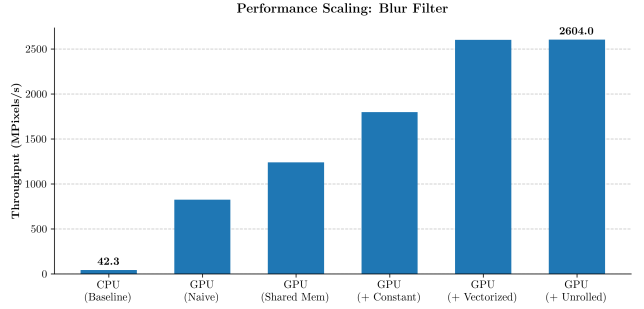
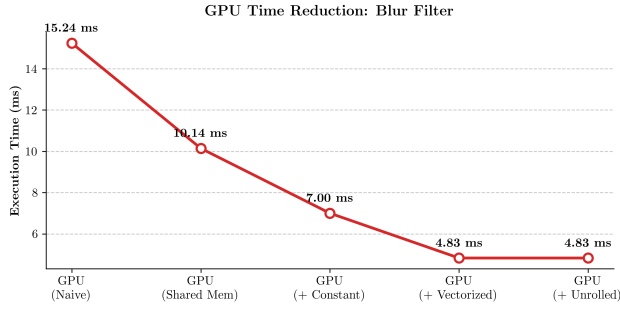
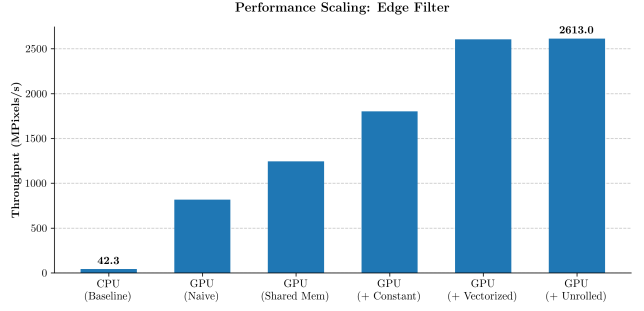
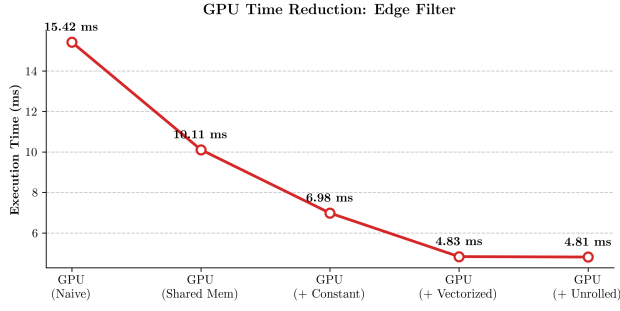
Kernel	Time (ms)	Throughput (MP/s)	Speedup
CPU (Baseline)	297.30	42.32	1.00x
GPU (Naive)	15.24	825.71	19.51x
GPU (Shared Mem)	10.14	1240.50	29.32x
GPU (+ Constant)	7.00	1798.14	42.49x
GPU (+ Vectorized)	4.83	2602.19	61.50x
GPU (+ Unrolled)	4.83	2604.02	61.54x

Table 4: Performance Summary - **Blur Filter**

Kernel	Time (ms)	Throughput (MP/s)	Speedup
CPU (Baseline)	296.86	42.38	1.00x
GPU (Naive)	15.45	814.44	19.22x
GPU (Shared Mem)	10.14	1240.99	29.28x
GPU (+ Constant)	7.01	1794.87	42.35x
GPU (+ Vectorized)	4.84	2600.06	61.35x
GPU (+ Unrolled)	4.83	2603.16	61.43x

Table 5: Performance Summary - **Sharpen Filter**

The following figures show the impact of each optimization in terms of execution time and throughput.



10 Roofline model

The roofline model is a tool to understand if the system is compute bound or memory bound. It provides a visual performance bound combining into one graph the peak compute performance and the peak memory bandwidth. The graph is built retrieving the computational roof and the bandwidth roof from the hardware data-sheet. The arithmetic (or computational) intensity is defined as:

$$AI = \frac{\text{Operations}}{\text{ByteTransferred}} \quad [FLOPS/B] \quad (6)$$

A low arithmetic intensity indicates that more time is spent moving data than performing computations, shifting the workload into the memory-bound region while a high arithmetic intensity means the opposite. The following roofline analysis is based on an analytical estimate of the data transferred by each block. The estimate assumes that an entire 18×18 RGBA tile is read from global memory and that a 16×16 RGBA output tile is written. Therefore, the results should be interpreted as an approximation rather than as a direct measurement. See Section 12 for further details on profiling.

In RGBA space each pixel has 4 values. For the last optimized kernel, it is possible to assume that the global memory accesses for each block are:

- Reading: $18 \times 18 \times 4 \text{ Byte} = 1296 \text{ Byte}$
- Writing: $16 \times 16 \times 4 \text{ Byte} = 1024 \text{ Byte}$

For a total of $1024 + 1296 = 2320$ Byte. Given that the active pixels are 256, the byte transferred for each pixel are:

$$\frac{2320}{256} \approx 9 \text{ B/pixel}$$

Counting each multiply and add as one FLOP, for each output pixel: $(9 \text{ mult} + 9 \text{ sums}) \times 3 \text{ channel} = 54 \text{ FLOP}$. The number of arithmetic operations for each moved byte is:

$$AI_{kernel} = \frac{54}{9} \approx 6 \text{ FLOP/B}$$

Given the target device specifications²:

$$P_{peak} = 1.13 \text{ TFLOP/s}$$

$$BW_{peak} = 48 \text{ GB/s}$$

It is possible to compute the position of the decision boundary:

$$AI_{ridge} = \frac{P_{Peak}}{BW_{Peak}} = \frac{1.13 \text{ TFLOP/s}}{48 \text{ GB/s}} \approx 23.5 \text{ FLOP/B}$$

Since:

$$AI_{kernel} < AI_{ridge} \rightarrow 6 < 23.5 \rightarrow \textbf{Memory Bound}$$

According to the analytical Roofline model, the kernel operates in the memory-bound region. Given the $AI_{kernel} = 6 \text{ FLOP/B}$, it is possible to compute the attainable performance roof, that is the maximum performance reachable by the system:

$$AI_{kernel} \times BW_{peak} = 6 \text{ FLOP/B} \times 48 \text{ GB/s} = 288 \text{ GFLOP/s}$$

To compute the achieved performance the effective memory bandwidth must be considered. As already mentioned, the resolution of the input image used for the test is 4344×2896 with four channels (RGBA). Given a block size of 16×16 , the number of blocks is

$$\lceil \frac{4344}{16} \rceil \times \lceil \frac{2896}{16} \rceil = 272 \times 181 = 49\,232 \text{ Blocks}$$

The transferred data for each block is 2320 Byte. The transferred data for the whole image is

$$49\,232 \text{ Blocks} \times 2320 \text{ Byte} \approx 0.114 \text{ GB}$$

The last optimized kernel has an execution time of $4.81 \text{ ms} = 0.00481 \text{ s}$ (Table 3). Consequently the effective bandwidth according to the analytical memory-traffic model is

$$BW_{kernel} = \frac{0.114 \text{ GB}}{0.00481 \text{ s}} \approx 23.7 \text{ GB/s}$$

Finally it is possible to compute the estimated achieved performance:

$$AI_{kernel} \times BW_{kernel} = 6 \text{ FLOP/B} \times 23.7 \text{ GB/s} \approx 142 \text{ GFLOP/s}$$

Figure 8 shows the roofline model.

²NVIDIA GeForce GT 1030

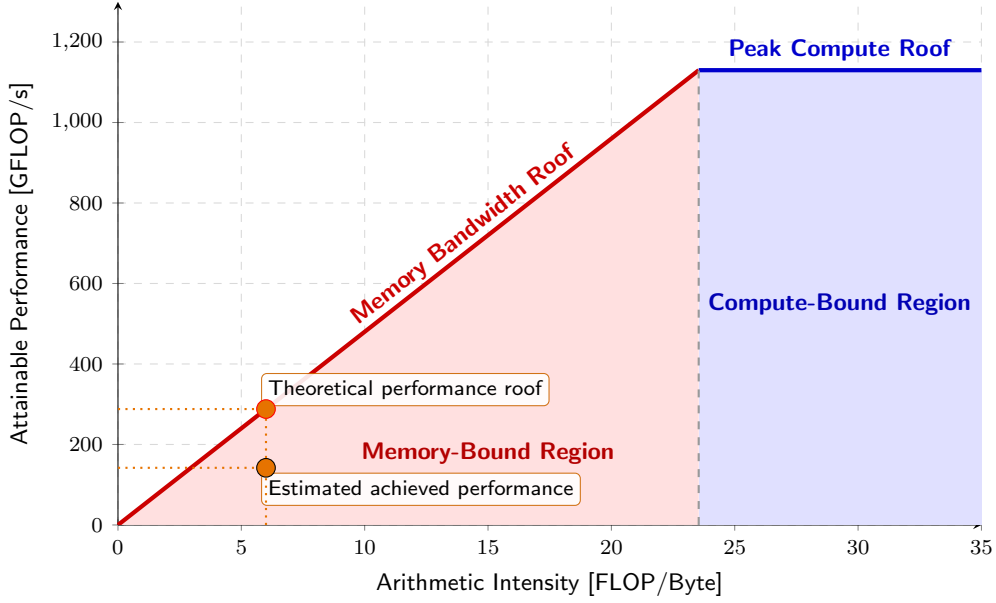


Figure 8: Analytical Roofline Model for the NVIDIA GeForce GT 1030 highlighting the 2D Convolution Kernel performance upper-bound.

The ratio between the estimated achieved performance (142 GFLOP/s) and the theoretical performance roof (288 GFLOP/s) is approximately 49%. The gap between the estimated achieved performance and the theoretical roof can be explained by several factors:

- The vectorized implementation transfers four bytes per pixel, but only the three RGB channels contribute to the convolution. The alpha channel therefore contributes to memory traffic without generating useful FLOPs.
- The model assumes that the complete tile is transferred from global memory for every block. In practice, L1/L2 cache hits may reduce the amount of data fetched from DRAM.
- The peak device bandwidth and peak compute performance are theoretical hardware limits and do not necessarily correspond to the sustained values achieved during this workload.
- The Roofline model counts only the arithmetic operations of the convolution. Other instructions, including address calculations, boundary checks, type conversions, and clamping, also consume execution resources without being represented as FLOPs in the model.

11 Conclusion

This work has a clearly educational nature, but the results are consistent and reproducible under the stated benchmark configuration. The incremental implementation shows that the optimization process is not mechanical but it concerns the hardware, the domains and the amount of data to be processed. Looking at Section 9, the three filters exhibit nearly identical execution times because they share the same computational structure: the filter size, number of channels, memory access pattern, and number of arithmetic operations per output pixel are the same. Only the filter coefficients differ, so the following analyses are valid for each filter. The first GPU implementation leads to a huge execution time reduction thanks to the introduction of parallel multi-thread execution. The shared and constant memory implementations are effective since they reduce the global memory traffic. The introduction of the vectorized accesses favors the memory coalescing and allows exploiting better the memory bandwidth. Then, as expected, the loop unrolling kernel does not bring significant improvement due to the optimization already introduced by the compiler. The Roofline model, shown in Section 10, is purely theoretical, but the larger gains obtained from memory-oriented optimizations are qualitatively consistent with the analytical roofline model. Overall, the fully optimized GPU kernel executes about $61\times$ faster than the serial CPU implementation, excluding host-device data transfer and memory allocation overhead. This demonstrates the effectiveness of a properly designed GPU memory hierarchy for this class of problems.

12 Future Developments

Given the significant results, this work could be a solid base, but as with any project, it presents some limitations. The objective of this final section is to explore different future developments that can be implemented.

- The Roofline model computed in Section 10 is purely theoretical and based on strong assumptions. The memory-bound conclusion has been reached taking the device specifications, estimating the transferred byte and the number of operations. The NVIDIA Nsight Compute profiler allows comparing the theoretically predicted results against the actual measurements.
- The device used for the benchmark is an entry-level GPU from 2017, with a Pascal architecture. Modern GPUs have larger memory bandwidth and more computing power; consequently, the arithmetic intensity threshold changes and the Roofline analysis could lead to a different conclusion.
- The serial CPU implementation is a valid baseline but to get an objective term of comparison already existing libraries can be adopted for the benchmark. The comparison with a professional library gives a more realistic reference about the reached performance.
- The test has been done with a fixed filter size: 3×3 . To get a complete summary about the performance one should test the program with larger filters. Consider a fixed block size 16×16 and a filter $F \times F$ with a radius $r = \lfloor F/2 \rfloor$. The number of active threads is $16^2 = 256$, the shared memory tile size is $(16 + 2r)^2$ and the number of halo pixels is $(16 + 2r)^2 - 256 = 4r^2 + 64r$. Consequently the ratio between the halo and the active pixels is $\frac{Halo}{Active} = \frac{4r^2 + 64r}{256}$.

$$- \text{ If the filter matrix is } 3 \times 3, r = 1 \rightarrow \frac{Halo}{Active} = \frac{68}{256} \approx 0.27$$

$$- \text{ If the filter matrix is } 5 \times 5, r = 2 \rightarrow \frac{Halo}{Active} = \frac{144}{256} \approx 0.56$$

$$- \text{ If the filter matrix is } 7 \times 7, r = 3 \rightarrow \frac{Halo}{Active} = \frac{228}{256} \approx 0.89$$

It is clear that with a larger filter, the shared memory halo of each tile becomes larger, the ratio between the halo pixels and the effective active threads increases. The GPU performs many redundant memory transactions. This overhead could produce a bottleneck.

- The test has been done with a fixed block size: 16×16 . Even if this block size is very common in this type of domain, the optimal block size should be determined experimentally. The block dimension has an impact on the shared memory tile size, consequently the ratio between the halo pixels and the effective active pixels changes. Consider a fixed filter size 3×3 , with a filter radius $r = 1$ and a block $B \times B$. The number of active threads is B^2 , the shared tile size is $(B + 2)^2$ and the number of halo pixels is $(B + 2)^2 - B^2 = 4B + 4$. Consequently the ratio between the halo and the active pixels is $\frac{Halo}{Active} = \frac{4B + 4}{B^2}$.

$$- \text{ If the block size is } 8 \times 8 \rightarrow \frac{Halo}{Active} = \frac{36}{64} \approx 0.56$$

$$- \text{ If the block size is } 16 \times 16 \rightarrow \frac{Halo}{Active} = \frac{68}{256} \approx 0.27$$

$$- \text{ If the block size is } 32 \times 32 \rightarrow \frac{Halo}{Active} = \frac{132}{1024} \approx 0.13$$

It is clear that, given a fixed filter size, with larger blocks the ratio gets better but the shared memory to be allocated per block increases. Moreover, if the image size is not multiple of block dimension, the block on the border will contain inactive threads. This phenomenon could become more problematic with larger block sizes.

- The test has been done using a single high-resolution image (4344×2896). To get a complete summary about the performance one should test the program with different image sizes.

Bibliography

- Notes on Parallel Programming (2025) by Francesco Leporati
- Notes on Advanced Computer Architecture (2026) by Emanuele Torti
- Notes on Machine Learning (2025) by Claudio Cusano
- CUDA Programming Guide, Release 13.3, NVIDIA Corporation (May 27, 2026)
- NVIDIA GeForce GT 1030 documentation
(<https://www.nvidia.com/en-us/geforce/graphics-cards/gt-1030/>)
- TechPowerUp GPU Database, 'NVIDIA GeForce GT 1030 Specs', 2017
(<https://www.techpowerup.com/gpu-specs/geforce-gt-1030.c2954>)
- Mouna Afif, Yahia Said, Mohamed Atri, " Efficient 2D Convolution Filters Implementations on Graphics Processing Unit Using NVIDIA CUDA ", International Journal of Image, Graphics and Signal Processing(IJIGSP), Vol.10, No.8, pp. 1-8, 2018.DOI: 10.5815/ijigsp.2018.08.01
- stb repository (<https://github.com/nothings/stb>)
- Test Image: Sebastian Pena Lambarri - Whaleshark, Maldives

Declaration on AI Usage

As this project represents a first practical exposure to GPU parallel programming and CUDA architecture, generative AI tools were consulted as an educational guide. They were used to clarify theoretical concepts and to assist in writing LaTeX code for some illustrations. No narrative text was copied or automatically generated; all explanations, architectural analyses, and the entire codebase were authored, debugged, and benchmarked directly by the author.